Studium przypadku - analiza danych medycznych dotyczących kolki u koni

Systemy Uczące się

Autor: Adrian Kellner (141245)

Opis zbioru danych

Źródło

Oryginalne dane: http://archive.ics.uci.edu/dataset/47/horse+colic

Pobrane z: https://www.kaggle.com/datasets/yasserh/horse-survival-dataset

Oryginalny opis pochodzi z http://archive.ics.uci.edu/dataset/47/horse+colic. Zredagowałem go oraz, z powodu dużej objętości,uciąłem wiele eksperckich szczegółów o wpływie i istotności atrybutów na finalną decyzję (np. "im większy ból tym bardziej prawdopodobna potrzeba operacji" lub "... mało użyeczne ze względu na duże wahania"). Tekst pozostawiłem po angielsku, by słownictwo pozostało możliwie precyzyjne i brzmiało lepiej.

Atrybuty decyzyjne

W danych jest kilka mozliwych atrybutów do klasyfikacji:

- outcome: czy koń przeżył leczenie?
- surgical lesion: czy wymagana była operacja?
- surgical type: jakiego typu operacja była wymagana?

Najciekawsza i nabardziej użyteczna byłaby klasyfikacja jakiej operacji wymaga koń (z opcją "żadnej"), ale ze względu na małą ilość danych i ich wybrakowanie, klasyfikacja ta byłaby bardzo trudna. Dlatego chciałem się ograniczyć do klasyfikacji binarnej - czy operacja jest potrzebna, czy nie.

Opis atrybutów

1. **surgery - if horse had surgery - Yes / No**
2. age - adult / young (< 6 months)
3. hospital number - numeric id (may not be unique)
4. rectal temperature - linear - in C°.
5. pulse - linear - the heart rate in beats per minute
6. respiratory rate - linear
7. temperature of extremities - a subjective indication of peripheral circulation
    - possible values normal / warm / cool / cold
8. peripheral pulse - a subjective measurement
    - possible values are: normal / increased / reduced / absent
9. mucous membranes - a subjective measurement of colour
    - possible values are: normal pink / bright pink / pale pink / pale cyanotic / bright red / dark cyanotic
10. capillary refill time - a clinical judgement.
    - possible values: < 3 seconds / >= 3 seconds
11. pain - a subjective judgement of the horse's pain level
    - possible values: alert, no pain / depressed / intermittent mild pain / intermittent severe pain / continuous severe pain
    - should NOT be treated as a ordered or discrete variable!
12. peristalsis - an indication of the activity in the horse's gut.
    - possible values: hypermotile / normal / hypomotile / absent
13. abdominal distension
    - An IMPORTANT parameter.
    - possible values: none / slight / moderate / severe
14. nasogastric tube - this refers to any gas coming out of the tube
    - possible values: none / slight / significant
15. nasogastric reflux

- possible values: none / >1 liter / <1 liter

16. nasogastric reflux PH - linear - scale is from 0 to 14 with 7 being neutral
    - normal values are in the 3 to 4 range
17. rectal examination - feces
    - possible values: normal / increased / decreased / absent
18. abdomen
    - possible values: normal / other / firm feces in the large intestine / distended small intestine / distended large intestine
19. packed cell volume - linear - the # of red cells by volume in the blood

20. total protein - linear - Values are in g/dL

21. abdominocentesis appearance - a needle is put in the horse's abdomen and fluid is obtained from the abdominal cavity
    - possible values: clear / cloudy / serosanguinous
22. abdomcentesis total protein - linear - Values are in g/dL

23. **outcome - what eventually happened to the horse?**
    - **possible values: lived / died / was euthanized**
24. **surgical lesion - retrospectively, was the problem (lesion) surgical?**
    - **possible values: Yes / No**
25. 26, 27 type of lesion
    - first number is site of lesion {1 = gastric, 2 = sm intestine, 3 = lg colon, 4 = lg colon and cecum, 5 = cecum, 6 = transverse colon, 7 = retum/descending colon, 8 = uterus, 9 = bladder, 11 = all intestinal sites, 00 = none}
    - second number is type {1 = simple, 2 = strangulation, 3 = inflammation, 4 = other}
    - third number is subtype {1 = mechanical, 2 = paralytic, 0 = n/a}
    - fourth number is specific code {1 = obturation, 2 = intrinsic, 3 = extrinsic, 4 = adynamic, 5 = volvulus/torsion, 6 = intussuption, 7 = thromboembolic, 8 = hernia, 9 = lipoma/slenic incarceration, 10 = displacement, 00 = n/a}
26. cp_data - is pathology data present for this case? - possible values: Yes/No
    - this variable is of no significance since pathology data is not included or collected for these cases

Wstępna analiza

In [ ]:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

Przegląd wartości i ich rozkładów

In [ ]:

```
df = pd.read_csv('horse.csv')
```

In [ ]:

```
binary = ['surgery', 'age', 'cp_data', 'surgical_lesion']
numerical = ['rectal_temp', 'pulse', 'respiratory_rate', 'nasogastric_reflux_ph', 'packed_cell_volume', 'total_protein', 'abdomo_pr
rankable = ['temp_of_extremities', 'capillary_refill_time', 'peripheral_pulse', 'peristalsis', 'abdominal_distention', 'nasogastric
categorical = ['mucous_membrane', 'pain', 'rectal_exam_feces', 'abdomen', 'abdomo_appearance', 'outcome']
class_columns = ['surgical_lesion', 'lesion_1', 'lesion_2', 'lesion_3']
```

Przykładowa obserwacja

Wybrałem pierwszy przykład zawierający komplet cech (bez NaN). Jest to dorosły koń, który wymagał operacji, przeszedł ją i przeżył.

In [ ]:

```
df.dropna().iloc[0]
```

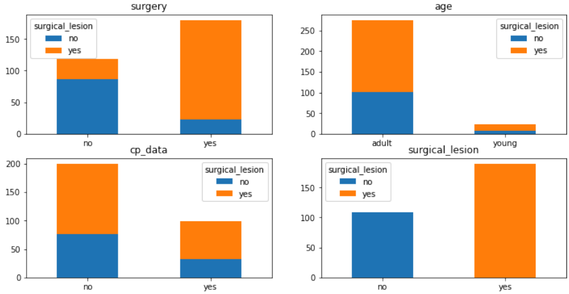
Out[ ]:

```
surgery                            yes
age                              adult
hospital_number                 528548
rectal_temp                       38.1
pulse                             66.0
respiratory_rate                  12.0
temp_of_extremities               cool
peripheral_pulse               reduced
mucous_membrane             bright_red
capillary_refill_time       less_3_sec
pain                         mild_pain
peristalsis                 hypomotile
abdominal_distention              none
nasogastric_tube                slight
nasogastric_reflux                none
nasogastric_reflux_ph              3.0
rectal_exam_feces            increased
abdomen                  distend_large
packed_cell_volume                44.0
total_protein                      6.0
abdomo_appearance               cloudy
abdomo_protein                     3.6
outcome                          lived
surgical_lesion                    yes
lesion_1                          2124
lesion_2                             0
lesion_3                             0
cp_data                            yes
Name: 10, dtype: object
```

Atrybuty binarne

In [ ]:

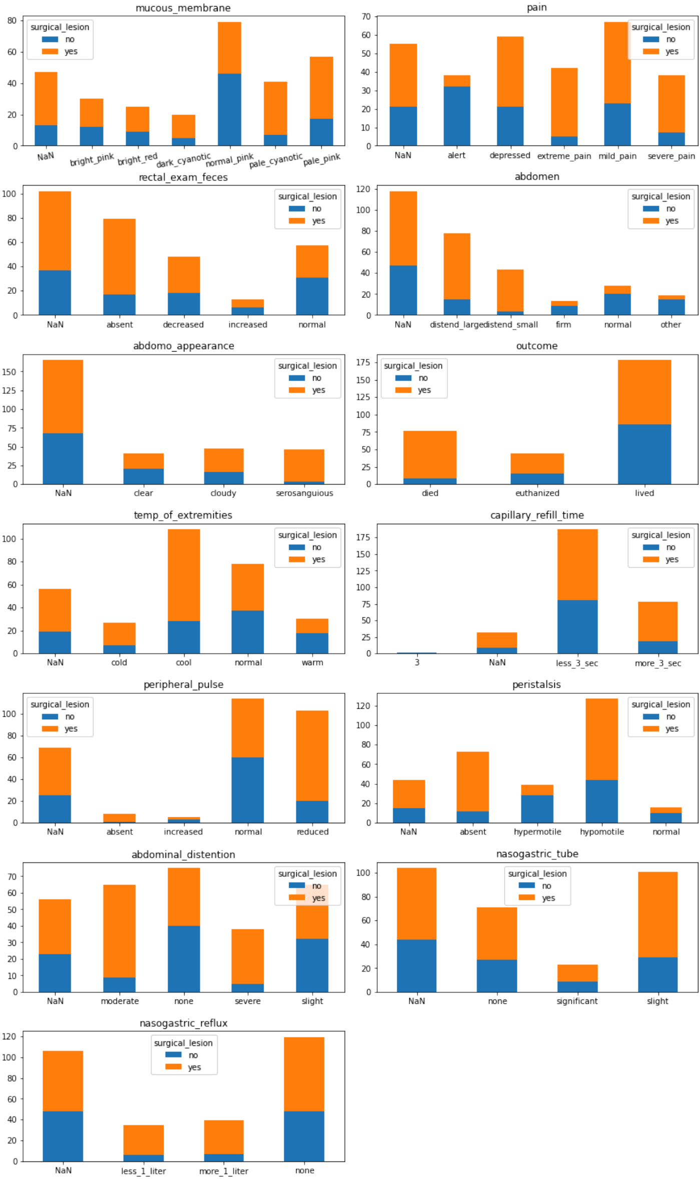
```python
fig, axs = plt.subplots(ncols=2, nrows=2, figsize=(12, 6))
axs = axs.reshape(-1)
i_tmp = None
for i, name in enumerate(binary):
    # sns.countplot(x=name, ax=axs[i], data=df, hue='surgical_lesion')
    counts = df.fillna("NaN").groupby([name, 'surgical_lesion']).size().unstack(fill_value=0)
    counts.plot(kind='bar', stacked=True, ax=axs[i])
    axs[i].set_title(name)
    axs[i].set_xticklabels(counts.index, rotation=0)
    axs[i].set_xlabel('')
    i_tmp = i
for j in range(i_tmp + 1, len(axs)):
    fig.delaxes(axs[j])
```



Atrybuty kategoryczne

In [ ]:

```python
fig, axs = plt.subplots(ncols=2, nrows=8, figsize=(15, 30))
fig.subplots_adjust(hspace=0.3, wspace=0.1)
axs = axs.reshape(-1)
i_tmp = None
for i, name in enumerate(categorical+rankable):
    counts = df.fillna("NaN").groupby([name, 'surgical_lesion']).size().unstack(fill_value=0)
    counts.plot(kind='bar', stacked=True, ax=axs[i])
    axs[i].set_title(name)
    axs[i].set_xlabel('')
    if i==0: axs[i].set_xticklabels(counts.index, rotation=10)
    else: axs[i].set_xticklabels(counts.index, rotation=0)
    i_tmp = i
for j in range(i_tmp + 1, len(axs)):
    fig.delaxes(axs[j])
```



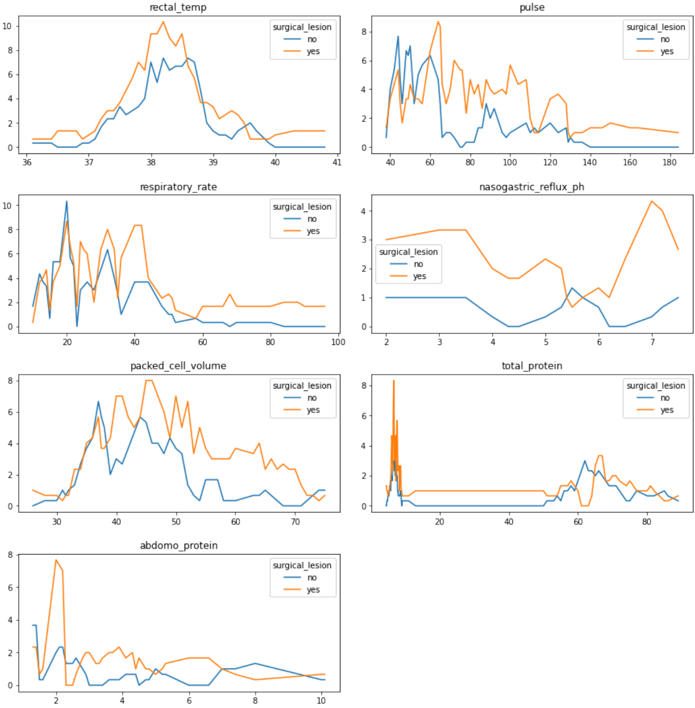
W niektórych atrybutach widać pojedyncze wartości, które wyraźniej wskazują na konieczność operacji:

- najwyższe poziomy bólu: extreme i severe (atrybut 'pain')
- spuchnięte (distend) jelito grube i cienkie w atrybucie 'abdomen' oraz wartości 'moderate' i 'severe' w 'abdomina_distention'
- serokrwawy (serosanguinous) płyn z podbrzusza (atrybut 'abdomo_appearance')
- wartość wyższa niż 0 w atrybucie 'nasogastric_reflux'

Atrybuty numeryczne

In [ ]:

```
fig, axs = plt.subplots(ncols=2, nrows=10, figsize=(15, 40))
fig.subplots_adjust(hspace=0.3, wspace=0.1)
axs = axs.reshape(-1)
i_tmp = None
for i, name in enumerate(numerical):
    counts = df.groupby([name, 'surgical_lesion']).size().unstack(fill_value=0)
    smoothed_counts = counts.rolling(window=3).mean()
    smoothed_counts.plot(kind='line', stacked=False, ax=axs[i])

    axs[i].set_title(name)
    axs[i].set_xlabel('')
    i_tmp = i

for j in range(i_tmp+1, len(axs)):
    fig.delaxes(axs[j])
```



Niestety wśród atrybutów numerycznych rozkłady wartości nie formują szczególnie wyraźnych skupisk, które rozdzielałyby klasę decyzyjną. Najbardziej użyteczna pod tym kątem może być informacja z wykresu 'pulse' na którym dla wartości 65 i większych następuje gwałtowny spadek decyzji 'no', a zatem u koni o podwyższonym pulsie znacznie bardziej wzrasta potrzeba operacji.

Z kolei na wykresie 'total_protein' zauważyłem podejrzanie odległe od siebie dwa skupienia wartości, jedno w zakresie 3,3-13 o wartościach z częścią dziesiętną, a drugie 46-89 o wartościach wyłącznie całkowitych. Jestem przekonany, że dane zawierają błąd polegający na braku przecinka - przykładowo zamiast 64 powinno być 6,4.

In [ ]:

```python
print(df.loc[df['total_protein'] < 20, 'total_protein'].min())
print(df.loc[df['total_protein'] < 20, 'total_protein'].max())
print(df.loc[df['total_protein'] > 20, 'total_protein'].min())
print(df.loc[df['total_protein'] > 20, 'total_protein'].max())
```

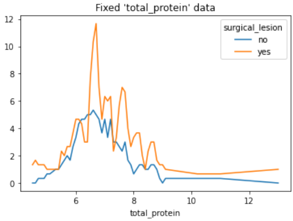
```
3.3
13.0
46.0
89.0
```

In [ ]:

```python
df.loc[df['total_protein'] > 20, 'total_protein'] /= 10

counts = df.groupby(['total_protein', 'surgical_lesion']).size().unstack(fill_value=0)
smoothed_counts = counts.rolling(window=3).mean()
ax = smoothed_counts.plot(kind='line')
ax.set_title("Fixed 'total_protein' data")
```

Out[ ]:

```
Text(0.5, 1.0, "Fixed 'total_protein' data")
```



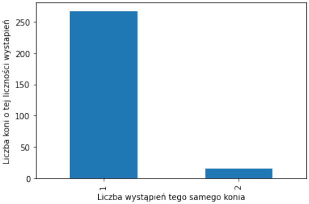
Atrybut 'hospital_number' pozwala nam zidentyfikować ponowne wystąpienia tego samego konia. Zbiór nie zawiera żadnych sygnatur czasowych, zatem nie jestem w stanie ułożyć danych chronologicznie i wykorzystywać dodatkowych historycznych danych do diagnozy (klasyfikacji). Nie chcę by klasyfikator uczył się na pamięć ID chorych koni, tylko ich objawów, dlatego usunę ten atrybut.

In [ ]:

```python
twice = df.hospital_number.value_counts().value_counts().T
ax = twice.plot(kind='bar')

ax.set_xlabel('Liczba wystąpień tego samego konia')
ax.set_ylabel('Liczba koni o tej liczności wystapień')
```

Out[ ]:

```
Text(0, 0.5, 'Liczba koni o tej liczności wystapień')
```



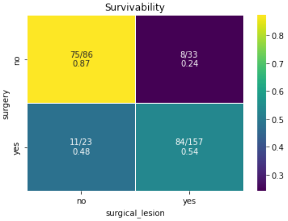
Na wykresie poniżej analizuję przeżywalność koni w zależności od tego czy potrzebowały operacji (oś X) oraz czy tę operację przeszły (oś Y). Wartości na wykresie pokazują stosunek liczby koni, które przeżyły (atrybut 'outcome') do całkowitej liczby koni w danej kategorii (lived/[lived+died+euthanized]).

In [ ]:

```python
df['outcome_binary'] = df['outcome'].apply(lambda x: 1 if x == "lived" else 0)
outcome = df.groupby(['surgery', 'surgical_lesion'])['outcome_binary'].mean().unstack()

pivot_table = df.pivot_table(index='surgery', columns='surgical_lesion', values='outcome_binary', aggfunc='sum', fill_value=0)
total_counts = df.groupby(['surgery', 'surgical_lesion']).size().unstack(fill_value=0)
labels = (pivot_table.astype(str) + '/' + total_counts.astype(str) + '\n' + outcome.apply(lambda x: x.round(2)).astype(str)).values

fig, ax = plt.subplots()
sns.heatmap(outcome, annot=labels, fmt='', cmap='viridis', linewidths=.5, ax=ax)
ax.set_title('Survivability')
```

Out[ ]:

```
Text(0.5, 1.0, 'Survivability')
```



surgery \ surgical_lesion	no	yes
no	75/86 0.87	8/33 0.24
yes	11/23 0.48	84/157 0.54

Z wykresu widać, że w przypadku tego zbioru danych konie, którym **przeprowadzono operację** mają szanse przeżycia bliską 50%, niezależnie od tego, czy tej operacji potrzebowały (`surgical_lesion`). Jednak wykonanie operacji, gdy koń jej nie potrzebuje, znacznie zmniejsza przeżywalność (z 87% do 48%) i znacznie zwiększa, gdy potrzebuje (z 24% do 54%).

Choć atrybut `outcome` może dodawać istotną informację i być bardzo skorelowany z klasą, to z punktu widzenia użyteczności klasyfikatora, jego użycie jest niepraktyczne. Idealnie byłoby wykorzystywać klasyfikator przed śmiercią konią, dlatego atrybut ten usunę z danych. Analogicznie niepraktyczny jest atrybut 'surgery', który jest pochodną klasy decyzyjnej.

Wedle opisu danych 'cp_data' powinno być nieistotne, jednak brak szegółów dotyczących patogenu nie oznacza, że sama informacja o wystąpieniu patogenu nie będzie bez znaczenia.

W atrybucie `capillary_refill_time` uwzględniona jest wartość "dokładnie 3" [sekundy] i pojawia sie ona w bardzo małej liczbie przypadków, więc postanowiłem połączyć "3" i "more_3_sec" w jedną kategorię "noless_3_sec".

In [ ]:

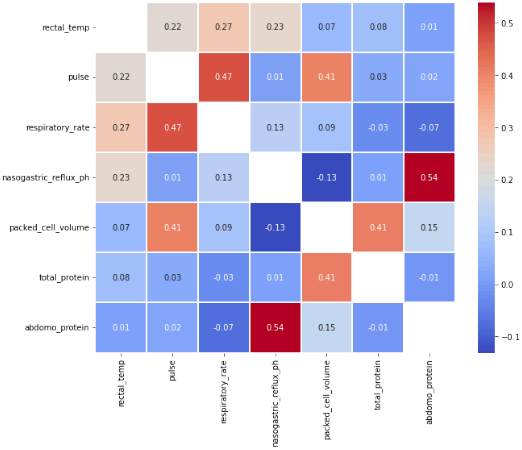
```
df.drop(columns=['hospital_number', 'outcome', 'surgery'], inplace=True)

df['capillary_refill_time'] = df['capillary_refill_time'].replace(['3', 'more_3_sec'], 'noless_3_sec')
```

Analiza zależności

Korelacja

In [ ]:

```
correlation_matrix = df[numerical].corr()
np.fill_diagonal(correlation_matrix.values, np.nan)

plt.figure(figsize=(10, 8))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt=".2f", linewidths=0.4)
plt.show()
```



	rectal_temp	pulse	respiratory_rate	nasogastric_reflux_ph	packed_cell_volume	total_protein	abdomo_protein
rectal_temp		0.22	0.27	0.23	0.07	0.08	0.01
pulse	0.22		0.47	0.01	0.41	0.03	0.02
respiratory_rate	0.27	0.47		0.13	0.09	-0.03	-0.07
nasogastric_reflux_ph	0.23	0.01	0.13		-0.13	0.01	0.54
packed_cell_volume	0.07	0.41	0.09	-0.13		0.41	0.15
total_protein	0.08	0.03	-0.03	0.01	0.41		-0.01
abdomo_protein	0.01	0.02	-0.07	0.54	0.15	-0.01	

In [ ]:

```
from scipy import stats
bi_corr = df[numerical].corrwith(df['surgical_lesion'].map({'no':0, 'yes':1}), method=stats.pointbiserialr)
bi_corr.index = ['Correlation', 'P-value']
bi_corr
```

Out[ ]:

	rectal_temp	pulse	respiratory_rate	nasogastric_reflux_ph	packed_cell_volume	total_protein	abdomo_protein
Correlation	-0.018796	0.269318	0.239740	0.002079	0.172737	0.082523	0.089305
P-value	0.772514	0.000006	0.000172	0.988210	0.004419	0.179642	0.374481

Wstępne przetwarzanie

Brakujące wartości

In [ ]:

```
pd.DataFrame({
    'NaN Count': df.isna().sum(),
    'NaN Percentage': (df.isna().mean() * 100).round(2)
})
```

Out[ ]:

	NaN Count	NaN Percentage
age	0	0.00
rectal_temp	60	20.07
pulse	24	8.03
respiratory_rate	58	19.40
temp_of_extremities	56	18.73
peripheral_pulse	69	23.08
mucous_membrane	47	15.72
capillary_refill_time	32	10.70
pain	55	18.39
peristalsis	44	14.72
abdominal_distention	56	18.73
nasogastric_tube	104	34.78
nasogastric_reflux	106	35.45
nasogastric_reflux_ph	246	82.27
rectal_exam_feces	102	34.11
abdomen	118	39.46
packed_cell_volume	29	9.70
total_protein	33	11.04
abdomo_appearance	165	55.18
abdomo_protein	198	66.22
surgical_lesion	0	0.00
lesion_1	0	0.00
lesion_2	0	0.00
lesion_3	0	0.00
cp_data	0	0.00
outcome_binary	0	0.00

W zbiorze brakuje sporej części danych, bo od 8% do 66% na 19 atrybutach.

We wszystkich atrybutach kategorycznych dodam kategorię 'Unknown', a numeryczne wypełnię stosując `KNNImputer` z biblioteki `sklearn.impute` dodając jednocześnie atrybut informujący, że dana była wybrakowana.

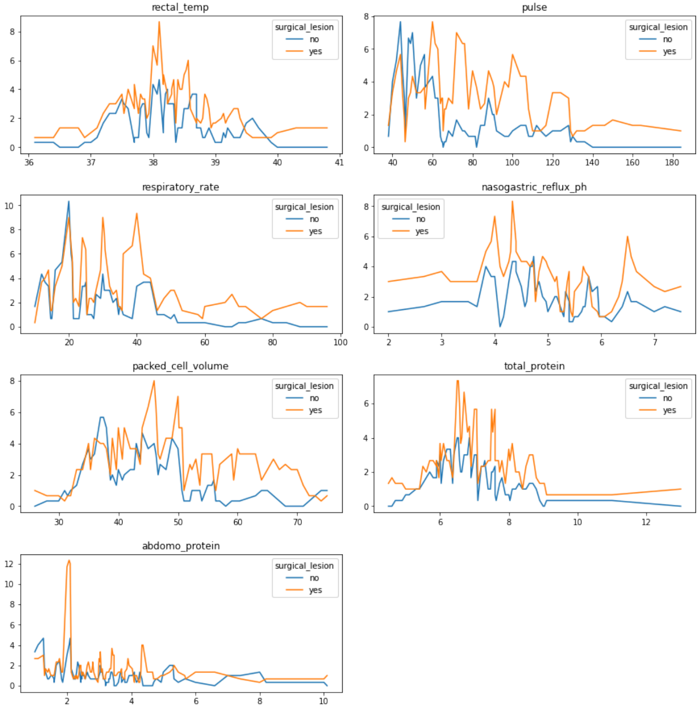
Uzupełnione dane numeryczne

In [ ]:

```
from sklearn.compose import ColumnTransformer
from sklearn.impute import KNNImputer
num_preprocessor = ColumnTransformer(transformers=[("num", KNNImputer(n_neighbors=3), numerical)])
num_transformed = pd.DataFrame(num_preprocessor.fit_transform(df[numerical]), columns=numerical)
df_transformed = df.copy()
df_transformed[numerical] = num_transformed
```

In [ ]:

```
fig, axs = plt.subplots(ncols=2, nrows=10, figsize=(15, 40))
fig.subplots_adjust(hspace=0.3, wspace=0.1)
axs = axs.reshape(-1)
i_tmp = None
for i, name in enumerate(numerical):
    counts = df_transformed.groupby([name, 'surgical_lesion']).size().unstack(fill_value=0)
    smoothed_counts = counts.rolling(window=3).mean()
    smoothed_counts.plot(kind='line', stacked=False, ax=axs[i])

    axs[i].set_title(name)
    axs[i].set_xlabel('')
    i_tmp = i
for j in range(i_tmp+1, len(axs)):
    fig.delaxes(axs[j])
```



Klasyfikacja

Klasyfikację przeprowadzę z użyciem regresji logistycznej, SVM i XGBoost. Każdy model poddam 10-krotnej walidacji krzyżowej. Dane numeryczne są uzupełnione przez algorytm KNN, a kategoryczne zakodowane przy użyciu `OneHotEncoder`.

In [ ]:

```
from sklearn.metrics import make_scorer, roc_auc_score, accuracy_score, f1_score, mean_absolute_error, precision_score, recall_scor
from imblearn.metrics import geometric_mean_score

from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold, cross_validate
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler, OrdinalEncoder
from sklearn.metrics import confusion_matrix
from sklearn.svm import SVC
from xgboost import XGBClassifier
```

```
np.random.seed(0)
```

In [ ]:

```
binary = ['age', 'cp_data', ]
numerical = ['rectal_temp', 'pulse', 'respiratory_rate', 'nasogastric_reflux_ph', 'packed_cell_volume', 'total_protein', 'abdomo_pr
rankable = ['temp_of_extremities', 'capillary_refill_time', 'peripheral_pulse', 'peristalsis', 'abdominal_distention', 'nasogastric
categorical = ['mucous_membrane', 'pain', 'rectal_exam_feces', 'abdomen', 'abdomo_appearance']

X = df[binary + numerical + rankable + categorical]
y = df['surgical_lesion']
y = y.map({'no': 0, 'yes': 1, })
```

In [ ]:

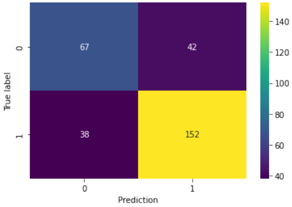
```
def make_pipeline(classif, num_indic=True):
    numeric_transformer = Pipeline(
        steps=[
            ("imputer", KNNImputer(n_neighbors=3, add_indicator=num_indic)),
            ("scaler", StandardScaler())]
    )
    binary_transformer = Pipeline(
        steps=[
            ("encoder", OrdinalEncoder()),
            # ("imputer", KNNImputer(n_neighbors=3)),
        ]
    )
    categorical_transformer = Pipeline(
        steps=[
            ("imputer", SimpleImputer(strategy="constant", fill_value="NA")),
            ("encoder", OneHotEncoder(handle_unknown="error"))]
    )
    preprocessor = ColumnTransformer(
        transformers=[
            ("num", numeric_transformer, numerical),
            ("bin", binary_transformer, binary),
            ("cat", categorical_transformer, categorical + rankable),
        ]
    )
    return Pipeline(steps=[("preprocessor", preprocessor), ("classifier", classif)])

scoring = {
    'g_mean': make_scorer(geometric_mean_score),
    'f1': make_scorer(f1_score),
    # 'accuracy': make_scorer(accuracy_score),
    'precision': make_scorer(precision_score),
    # 'recall': make_scorer(recall_score)
    'roc_auc': make_scorer(roc_auc_score)
}

def get_scores(res, scoring):
    scores = [res[f'test_{s}'] for s in list(scoring.keys())]
    mean = np.mean(scores, axis=1)
    std = np.std(scores, axis=1)
    out = pd.DataFrame({'mean': mean, 'std': std}, index=scoring.keys())
    return out

def plot_CV_heatmap(res):
    conf_matrix = None
    for i in range(10):
        X_test, y_test = X.loc[res['indices']['test'][i]], y.loc[res['indices']['test'][i]]
        predictions = res['estimator'][i].predict(X_test)
        if conf_matrix is None: conf_matrix = confusion_matrix(y_test, predictions)
        else: conf_matrix += confusion_matrix(y_test, predictions)
    sns.heatmap(conf_matrix, cmap='viridis', annot=True, fmt=".0f")
    plt.xlabel('Prediction')
    plt.ylabel('True label')

cv = StratifiedKFold(n_splits=10)
```

Regresja Logistyczna

In [ ]:

```
model = make_pipeline(LogisticRegression())
res1 = cross_validate(model, X, y, cv=cv, n_jobs=4, scoring=scoring, return_estimator=True, return_indices=True)
print(get_scores(res1, scoring))
plot_CV_heatmap(res1)
```

```
               mean       std
g_mean     0.696850  0.031677
f1         0.789411  0.042746
precision  0.785194  0.025543
roc_auc    0.707273  0.031009
```

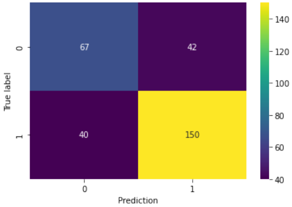


SVC

In [ ]:

```
model = make_pipeline(SVC(kernel='linear'))
res2 = cross_validate(model, X, y, cv=10, n_jobs=4, scoring=scoring, return_estimator=True, return_indices=True)
print(get_scores(res2, scoring))
plot_CV_heatmap(res2)
```

```
               mean       std
g_mean     0.689813  0.043695
f1         0.782502  0.048252
precision  0.784370  0.034664
roc_auc    0.701555  0.040172
```

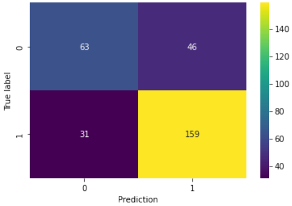


XGBoost

In [ ]:

```
model = make_pipeline(XGBClassifier())
res3 = cross_validate(model, X, y, cv=10, n_jobs=4, scoring=scoring, return_estimator=True, return_indices=True)
print(get_scores(res3, scoring))
plot_CV_heatmap(res3)
```

```
c:\Users\Adrian\anaconda3\lib\site-packages\joblib\externals\loky\process_executor.py:752: UserWarning: A worker stopped while some jobs were given to the executor. This can be caused by a too short worker timeout or by a memory leak.
  warnings.warn(
               mean       std
g_mean     0.681705  0.111386
f1         0.804863  0.043193
precision  0.782069  0.060156
roc_auc    0.707512  0.074430
```



Ogólna proporcja błędów na macierzach jest podobna na wszystkich trzech klasyfikatorach; ~50% True Positive i ~22% True Negative. Szczególnie zbliżone są przewidywania regresji i SVM, za to gradient boosting częściej przewiduje klasę '1' co sprowadza się do większej liczby zarówno True Positive jak i False Positive.

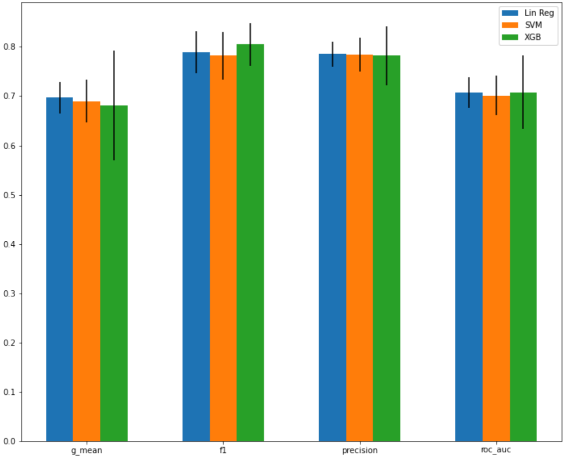
Porównanie

In [ ]:

```
width = 0.2
indices = np.arange(len(scoring))
fig, ax = plt.subplots(figsize=(12,10))
scores1 = get_scores(res1, scoring)
scores2 = get_scores(res2, scoring)
scores3 = get_scores(res3, scoring)
ax.bar(indices - width, scores1['mean'], width, yerr=scores1['std'], label='Lin Reg')
ax.bar(indices, scores2['mean'], width, yerr=scores2['std'], label='SVM')
ax.bar(indices + width, scores3['mean'], width, yerr=scores3['std'], label='XGB')
ax.set_xticks(indices)
ax.set_xticklabels(scoring.keys())
ax.legend()
```

Out[ ]:

```
<matplotlib.legend.Legend at 0x2d6249b2400>
```



Wnioski

- Modele mają bardzo zbliżone wyniki.
- SVM jest raczej najsłabszy, bo przewyższa jedynie XGBoost w metryce G-mean.
- Odchylenia wyników gradient boostingu są najwyższe, a regresji najniższe.
- Konie niepotrzebnie operowane miały mniejszą przeżywalność, co w opozycji do większości problemów klasyfikacji w temacie medycznym stwarza sytuację gdzie niekoniecznie dobrym jest nadmiernie klasyfikować odpowiednik etykiety "chory"